

Gerenciamento Avançado de Recursos com Manager Classes e Testes Automatizados no Android

Para exemplificar o uso dos gerenciadores de recursos do Android, foi desenvolvido um aplicativo simples que demonstra o uso do **WindowManager** com a criação de uma tela sobreposta à tela do aplicativo, o **PackageManager** para listar aplicativos instalados no AVD e o **ActivityManager** para obter informações de processos no sistema. O aplicativo conta com uma tela inicial onde utiliza uma estrutura de **RecyclerView** para listar as informações e a **TabLayout** para trocar a exibição das informações.

Objetivo

- Explicar as funções principais das Manager Classes no gerenciamento de atividades no Android;
- Implementar a interação entre Manager Classes e System Services no Android
- Utilizar o Espresso e JUnit como ferramentas de automação de testes para componentes gerenciados pelas Manager Classes.

Repositório

Link do Repositório: [gitHub](#)

Link da Aplicação: [Resource Management](#)



Nota Este link é da branch do repositório onde se encontra o código-fonte da aplicação. Após a avaliação da atividade, a branch será megeada na devel.

Sumário

1. [Ambiente de desenvolvimento](#)
 - 1.1. [Sistema Operacional](#)
 - 1.2. [Java\(JDK\)](#)
 - 1.3. [Android Studio](#)
 - 1.4. [Git](#)
 - 1.5. [Emulador Android](#)
2. [Resultado da Atividade](#)
 - 2.1. [Estrutura de pastas do repositório](#)
 - 2.2. [Implementação](#)
 - 2.2.1. [Ciclo de Vida da Activity](#)
 - 2.2.2. [Gerenciadores de Recursos](#)
 - 2.2.2.1. [PackageManager](#)
 - 2.2.2.2. [ActivityManager](#)
 - 2.2.2.3. [WindowManager](#)
 - 2.2.2.3.1. [Permissões](#)
 - 2.2.3. [Layout](#)

1. Ambiente de Desenvolvimento

1.1. Sistema Operacional

```
# O comando lsb_release imprime certas informações de LSB (Linux Standard Base) e distribuição.  
$ lsb_release -a  
No LSB modules are available.  
Distributor ID: Ubuntu  
Description: Ubuntu 22.04.5 LTS  
Release: 22.04  
Codename: jammy
```

1.2. Java(JDK)

```
# Retorna informações do Java caso esteja instalado  
$ java --version
```

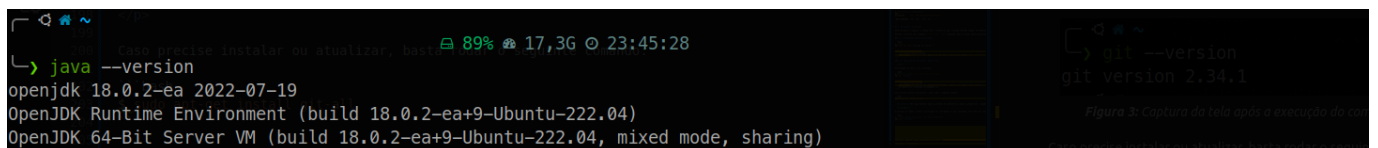


Figura 1: Captura da tela após a execução do comando, ferramenta instalada corretamente

1.3. Android Studio

```
# Informações gerais do Android Studio  
  
Android Studio Meerkat | 2024.3.1  
Build #AI-243.22562.218.2431.13114758, built on February 24, 2025  
Runtime version: 21.0.5+-12932927-b750.29 amd64  
VM: OpenJDK 64-Bit Server VM by JetBrains s.r.o.  
Toolkit: sun.awt.X11.XToolkit  
Linux 6.8.0-52-generic  
GC: G1 Young Generation, G1 Concurrent GC, G1 Old Generation  
Memory: 2968M  
Cores: 12  
Registry:  
  ide.experimental.ui=true  
  i18n.locale=  
Current Desktop: ubuntu:GNOME
```

1.4. Git

```
# Retorna a versão do Git caso esteja instalado  
$ git --version
```

```
└─> git --version
git version 2.34.1
```

Figura 2: Captura da tela após a execução do comando, neste caso ferramenta está instalada corretamente

1.5. Emulador Android

O Emulador Android utilizado é o mesmo da atividade anterior, não foi realiza que já foi entregue e descrita na sessão [1.5 Emulador Android](#) do relatório [Interface e Gerenciamento de Serviços no Android](#).

2. Resultado da Atividade

2.1 Estrutura de pastas do repositório

```
.
├── aosp
│   ├── kernel # Arquivos de configuração e patch do kernel do Android
│   └── src # Aplicativos desenvolvidos durante o curso
│       ├── apps # Aplicativos desenvolvidos até o momento
│       │   ├── AIDL_Interface
│       │   ├── audio_equalizer
│       │   └── resource_management # Projeto Hands on UA3 e UA4 da
│                                   disciplina: Gerenciamento Avançado de Recursos com Manager Classes e Testes
│                                   Automatizados no Android
│       ├── app
│       ├── build
│       ├── build.gradle.kts
│       ├── .gitignore
│       ├── .gradle
│       ├── gradle
│       ├── gradle.properties
│       ├── gradlew
│       ├── gradlew.bat
│       ├── .idea
│       ├── local.properties
│       └── settings.gradle.kts
├── docs
│   └── reports # Todos os relatórios no formato PDF
├── readme_files # Todos os relatórios escritos em markdown
│   ├── AIDL_dc3_u1_u2.md
│   ├── android_application_dc2_p1.md
│   ├── aosp_customization.md
│   ├── environmental_preparation.md
│   └── imgs
├── README.md # LEIA-ME principal do repositório
├── .vscode
│   └── settings.json
```

O Caminho do projeto desenvolvido conforme a estrutura de pastas acima é [Resource Management /aosp/src/apps/resource_management](#). Todos os arquivos do projeto do [android studio](#) estão dentro desta pasta.

2.2 Implementação

Conforme descrito no início do relatório, este aplicativo demonstra o uso de 3 gerenciadores de recursos do Android de maneira direta, sendo estes [PackageManager](#), [ActivityManager](#) e [WindowManager](#). As imagens abaixo apresentam a interface do aplicativo e posteriormente descreverei em detalhes a implementação e utilização dos recursos.

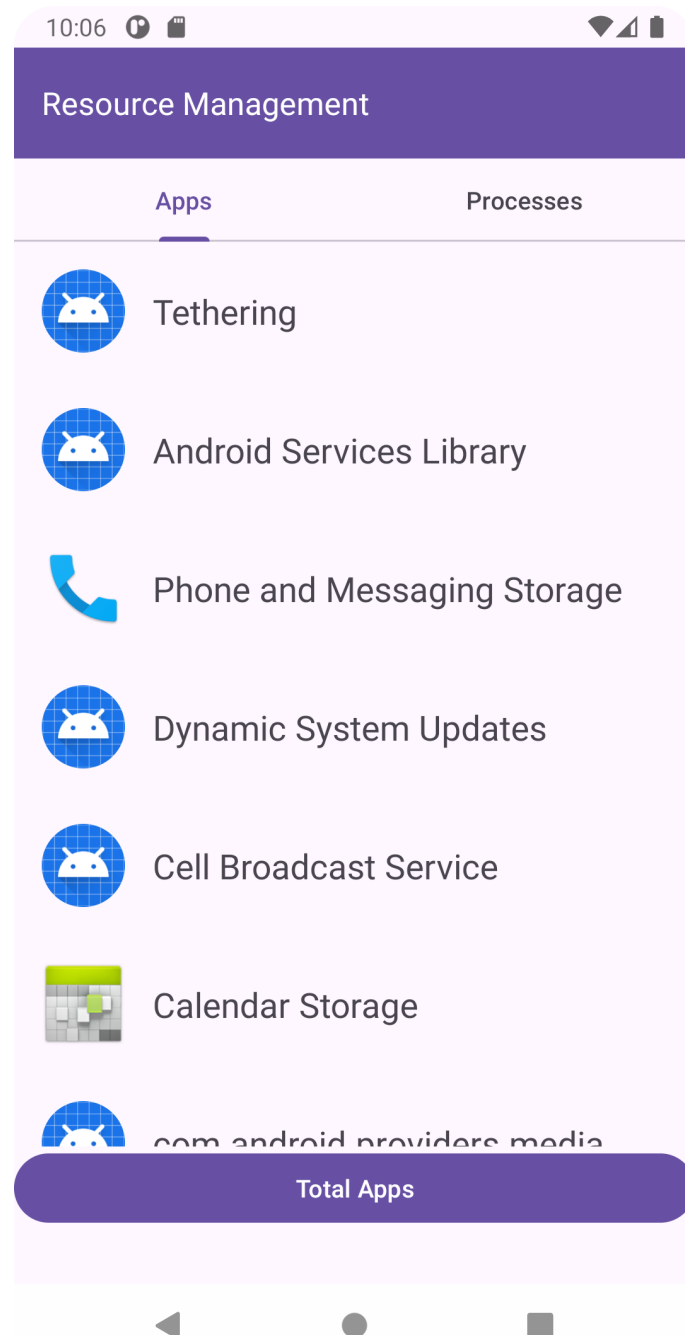


Figura 3: Tela inicial do aplicativo na tab de lista de Apps

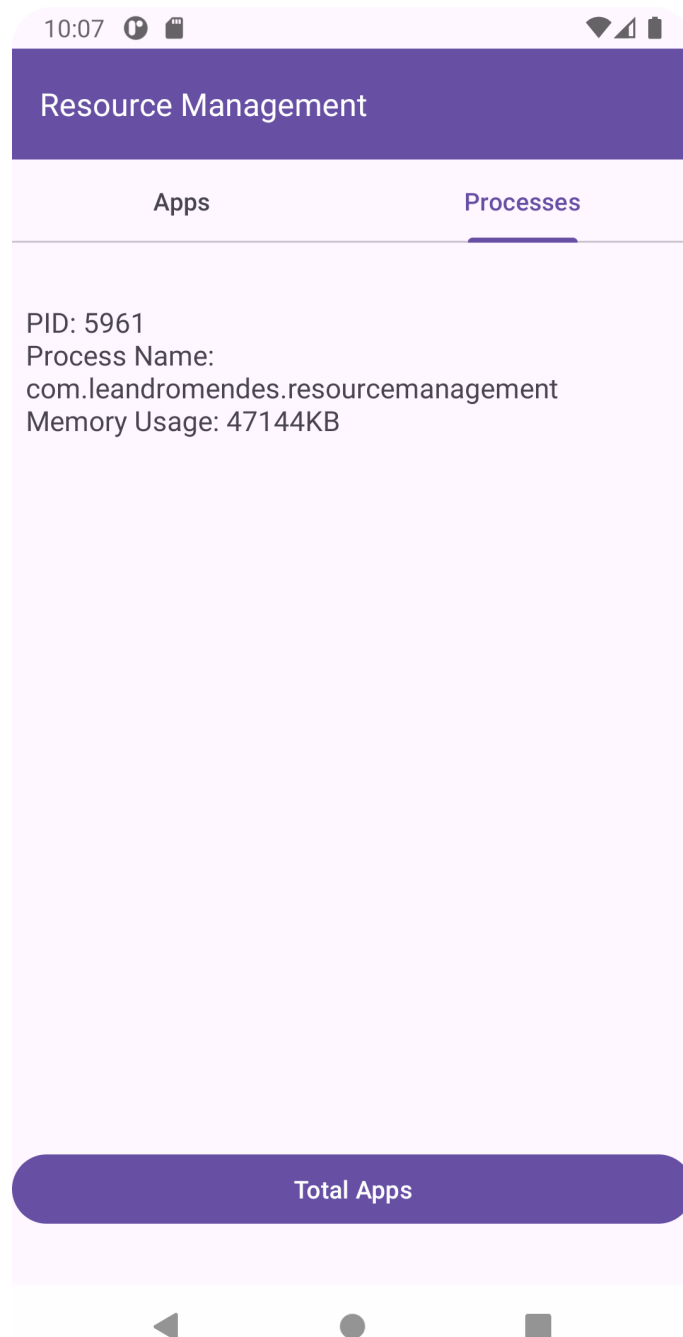


Figura 4: Tela do aplicativo na tab de lista de processos

A implementação do designer da tela foi realizada utilizando o [TabLayout](#) em conjunto com o widget [ViewPager2](#) onde foi adicionado como elemento de exibição uma [RecyclerView](#).

Uma das principais vantagens de se utilizar uma [recyclerView](#) para esse caso de uso é a sua flexibilidade e o baixo consumo de memória devido ao seu gerenciamento eficiente das Views.

2.2.1 Ciclo de Vida da Activity

O fluxo da Figura 5 apresenta o ciclo de vida de uma Activity de maneira simplificada.

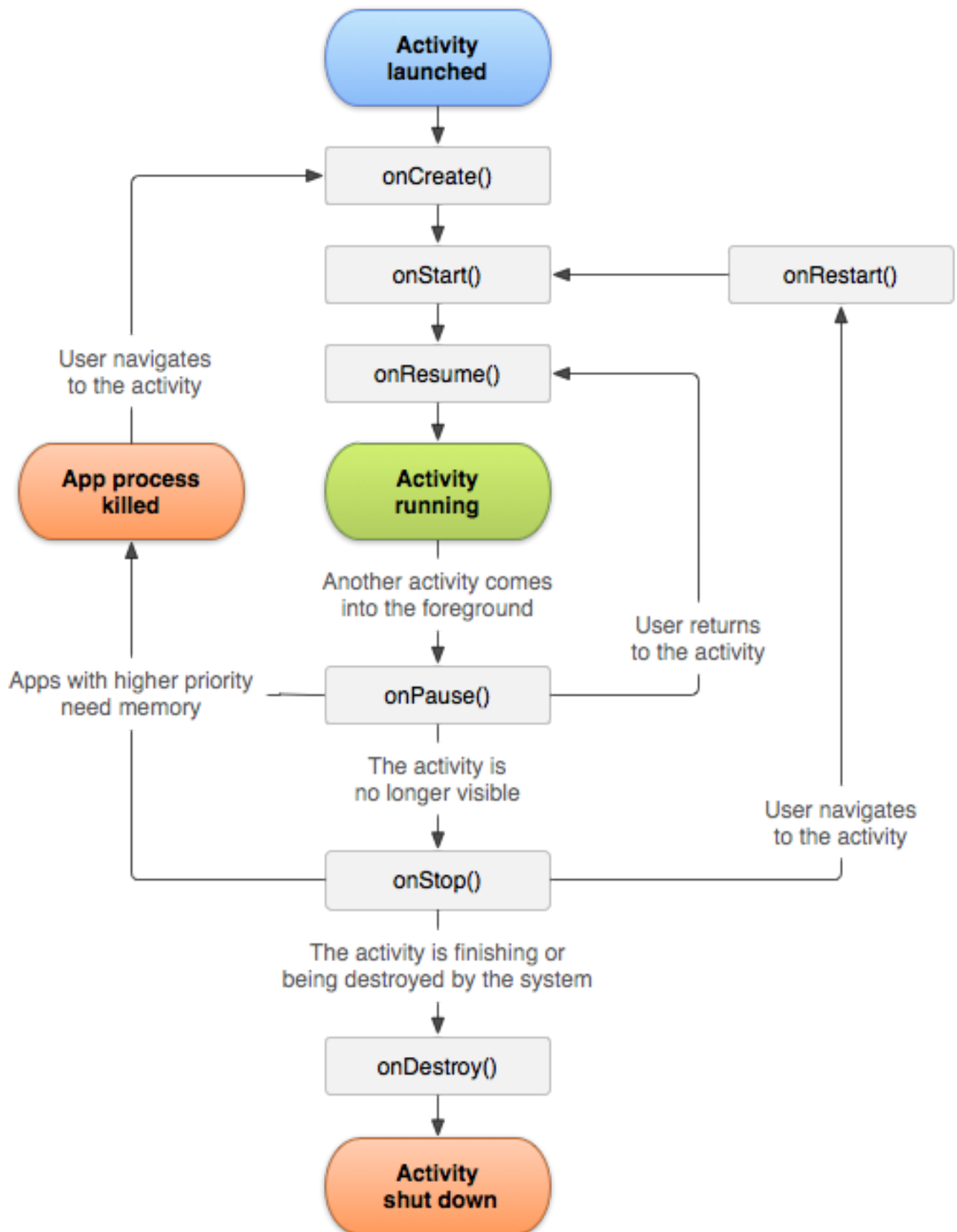


Figura 5: Ciclo de vida de uma Activity

Na Figura 6 temos os estados do ciclo de vida da nossa MainActivity, foi dada ênfase apenas nos estados que têm ação direta, os demais apenas log para fins de acompanhamento. Aqui é descrita a inicialização dos principais componentes da interface, nas próximas sessões irei explorar onde estão sendo utilizados os gerenciadores de recursos que citei anteriormente dentro do aplicativo.

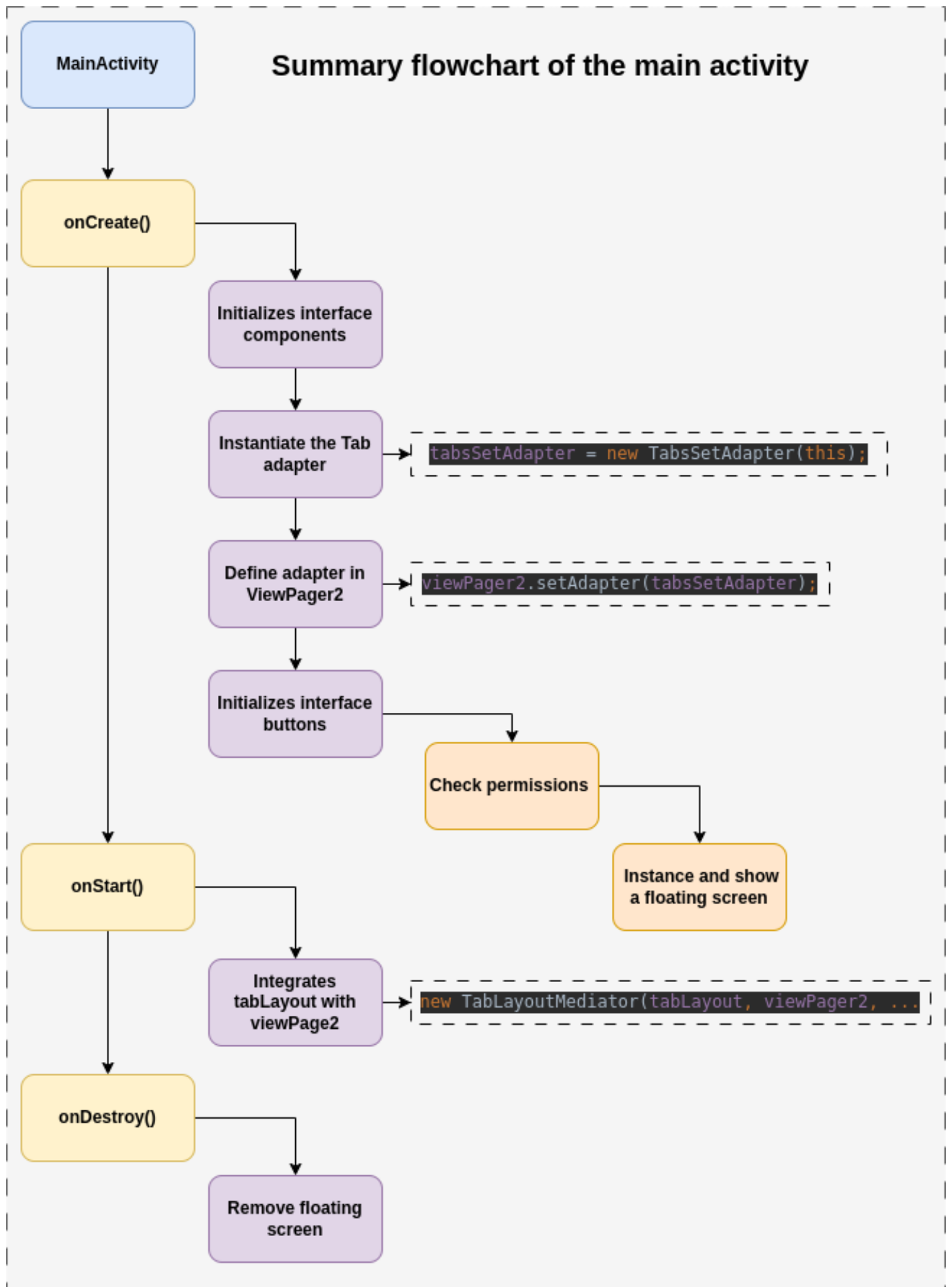


Figura 6: Tela do aplicativo na tab de lista de processos

A imagem abaixo mostra os logs sendo impressos no terminal, mostrando em que estado está a Activity durante sua execução.

```
--- PROCESS ENDED (5961) for package com.leandromendes.resourcemangement -----
--- PROCESS STARTED (6382) for package com.leandromendes.resourcemangement -----
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onCreate() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onStart() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onResume() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onPause() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onStop() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onStart() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onResume() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onPause() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onStop() called
6382-6382 ResourceMa...rLifecycle com...ndromendes.resourcemangement D onDestroy() called
```

Figura 7: Logs de execução

2.2.2 Gerenciadores de Recursos

2.2.2.1 PackageManager

O **PackageManager** foi utilizado para alimentar a **Tab** de lista de aplicativos, ele foi instanciado na classe **Appinfo** no método **GetAllAppsInfo()**.

```
public class AppInfo {

    /**
     * Gets list of installed applications
     * <p>
     * @param context Context of the activity
     * @return Returns list of installed applications
     */
    @NonNull
    public static List<ApplicationInfo> GetAllAppsInfo(@NonNull Context
context){
        PackageManager packageManager = context.getPackageManager();
        return
packageManager.getInstalledApplications(packageManager.GET_META_DATA);
    }
    ...
}
```

Este método retorna uma lista do tipo **ApplicationInfo** utilizada no método sobreposto (originado da classe **Fragment**) **onViewCreated** na classe **AppsTab** que estende **Fragment** e representa um fragmento da Tab.

```
public class AppsTab extends Fragment {

    ...
    @Override
```



```

    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        recyclerView.setLayoutManager(new
LinearLayoutManager(getContext()));

        List<ApplicationInfo> allAppsInstalled =
AppInfo.GetAllAppsInfo(getContext());
        AppViewAdapter appViewAdapter = new AppViewAdapter(getContext(),
allAppsInstalled);
        recyclerView.setAdapter(appViewAdapter);

    }
}

```

Neste mesmo método também é instanciado a **RecyclerView** que define seu adaptador (**AppViewAdapter**) que faz parte da sua implementação.

Aqui está o adaptador da **RecyclerView** que exibe a lista de aplicativos. Os métodos apresentados são os métodos mínimos da classe estendida (**RecyclerView.Adapter<AppViewAdapter.AppViewHolder>**) para o correto funcionamento da **RecyclerView**.

```

public class AppViewAdapter extends
RecyclerView.Adapter<AppViewAdapter.AppViewHolder> {

    private final List<ApplicationInfo> appList;
    private final PackageManager packageManager;

    /**
     * Provide a reference to the type of views that you are using
     * (custom ViewHolder)
     */
    public static class AppViewHolder extends RecyclerView.ViewHolder {
        private final ImageView appIcon;
        private final TextView AppName;

        public AppViewHolder(View itemView) {
            super(itemView);
            appIcon = itemView.findViewById(R.id.iconView);
            AppName = itemView.findViewById(R.id.appTextView);
        }
    }

    ...

    @NonNull
    @Override
    public AppViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        // Create a new view, which defines the UI of the list item
        View view = LayoutInflater.from(parent.getContext())

```

```

        .inflate(R.layout.application_item, parent, false);
        return new AppViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull AppViewHolder holder, int
position) {
        // Get element from your dataset at this position and replace the
        ApplicationInfo appInfo = appList.get(position);
        // contents of the view with that element
        holder.getAppName().setText(appInfo.loadLabel(packageManager));

        holder.getAppIcon().setImageDrawable(appInfo.loadIcon(packageManager));
    }

    // Return the size of your dataset (invoked by the layout manager)
    @Override
    public int getItemCount() {
        return appList.size();
    }
    ...

```

2.2.2.2 ActivityManager

A segunda Tab, que exibe uma **RecyclerView** de processos, tem uma estrutura parecida com a de lista de aplicativos mostrada na seção anterior, só que neste caso é utilizado o **ActivityManager** para obter informações sobre os processos. Na classe **ProcessInfo** o método **GetAllProcessInfo()** retorna uma lista do tipo **ActivityManager.RunningAppProcessInfo** obtida com o **activityManager** conforme trecho abaixo.

```

public class ProcessInfo {

    /**
     * Gets all the processes of the application that is running
     * <p>
     * @param context Context of the activity
     * @return Returns list of processes
     */
    @NonNull
    public static List<ActivityManager.RunningAppProcessInfo>
    GetAllProcessInfo(@NonNull Context context){
        ActivityManager activityManager = (ActivityManager)
        context.getSystemService(Context.ACTIVITY_SERVICE);
        return activityManager.getRunningAppProcesses();
    }
}

```

Este método é chamado na classe **ProcessTab** que é exatamente igual a **AppsTab**, o adaptador do **ProcessTab** é implementado na classe **ProcessViewAdapter** que é semelhante a **AppViewAdapter**

sendo diferente apenas como a View organiza as informações exibidas.

2.2.2.3 WindowManager

O **WindowManager** foi utilizado para exibir uma tela sobreposta à tela do aplicativo com o total de aplicativos instalados. Como pode ser visto na Figura 8, a interface possui um botão no canto inferior que, quando pressionado, exibe a tela flutuante.

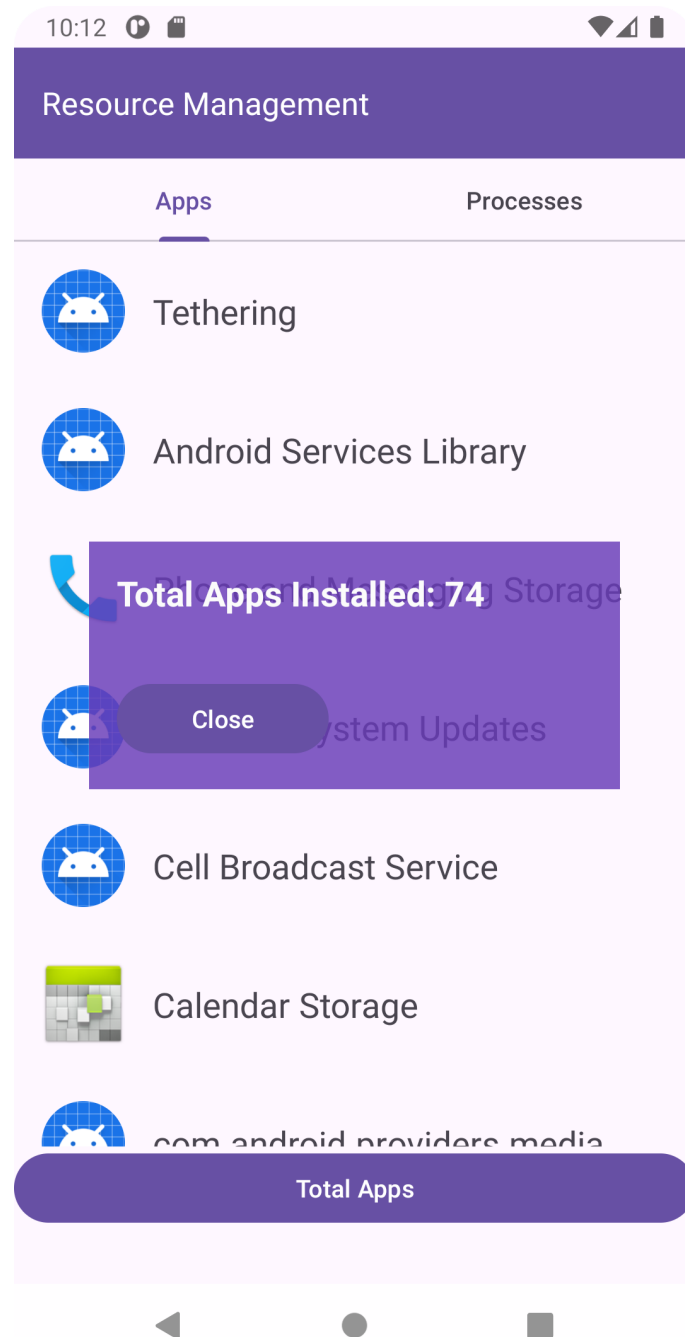


Figura 8: Tela Sobreposta

A instância do **windowManger** é obtida no método **onCreate()** da classe **MainActivity** assim como também é instanciada a tela sobreposta a uma View conforme trecho abaixo.

```
...  
@Override
```

```

protected void onCreate(Bundle savedInstanceState) {
    ...

    // Gets an instance of WindowManager to manage the floating window.
    windowManager = (WindowManager)
    getSystemService(Context.WINDOW_SERVICE);

    // Gets a LayoutInflater to inflate the layout of the floating window.
    LayoutInflater inflater = (LayoutInflater)
    getSystemService(Context.LAYOUT_INFLATER_SERVICE);
    infoFloatingView = inflater.inflate(R.layout.info_window, null);
    textView = infoFloatingView.findViewById(R.id.infoView);

    // Creates the layout parameters for the floating window
    params = new WindowManager.LayoutParams(
        WindowManager.LayoutParams.WRAP_CONTENT, // Window width
        adjusted to content
        WindowManager.LayoutParams.WRAP_CONTENT, // Window height
        adjusted to content
        WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY, // Layout
        type for overlay
        WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE, // Prevents the
        window from receiving incoming focus
        PixelFormat.TRANSLUCENT); // Makes the background of the window
        translucent

    // Sets the gravity of the floating window to center it on the screen
    params.gravity = Gravity.CENTER;
    ...

```

Quando o botão é pressionado, primeiro é feita uma verificação de permissão, caso não tenha, a mesma é solicitada ao usuário, após essa etapa a tela é adicionada à View e exibida conforme trecho abaixo.

```

private void showInfoWindow() {

    // Gets and set the total number of apps on the floating screen
    textView.setText("Total Apps Installed: " + GetTotalAppsInstall(this));

    // Add the floating window to the screen using WindowManager
    windowManager.addView(infoFloatingView, params);

    // Gets button that closes the overlay window
    Button closeButton = infoFloatingView.findViewById(R.id.closeButton);
    closeButton.setOnClickListener(v ->
    windowManager.removeView(infoFloatingView));
}

```

2.2.2.3.1 Permissões

Para que o `WindowManager` consiga exibir corretamente a tela sobreposta é necessária a permissão `<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW" />`. Esta permissão precisa ser adicionada ao arquivo `AndroidManifest.xml` conforme trecho abaixo.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"
/>
...

```

Por se tratar de uma permissão crítica, não basta apenas acrescentar ao arquivo de manifesto, é necessário ser concedida em tempo de execução. O trecho abaixo exibe o método responsável pela solicitação, o método é chamado toda vez que o botão é pressionado. Caso a permissão já tenha sido concedida, apenas exibe a tela, caso contrario requisita ação do usuário.

```
private void requestOverlayPermission() {
    if (!Settings.canDrawOverlays(this)) {
        Intent intent = new
Intent(Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
        Uri.parse("package:" + getPackageName()));
        // starts an Activity that will ask the user for permission.
        overlayPermissionLauncher.launch(intent);
    } else {
        // Permission already granted, create floating window
        showInfoWindow();
    }
}

```

```
private final ActivityResultLauncher<Intent> overlayPermissionLauncher =
    registerForActivityResult(new
ActivityResultContracts.StartActivityForResult(),
        result -> {
            if (Settings.canDrawOverlays(MainActivity.this)) {
                showInfoWindow();
            } else {
                Toast.makeText(MainActivity.this, "Permission
denied!",
                                Toast.LENGTH_SHORT).show();
            }
        });

```

Quando a permissão ainda não foi concedida, a tela da permissão da figura 9 é exibida, basta entrar no aplicativo `Resource Management` para conceder.

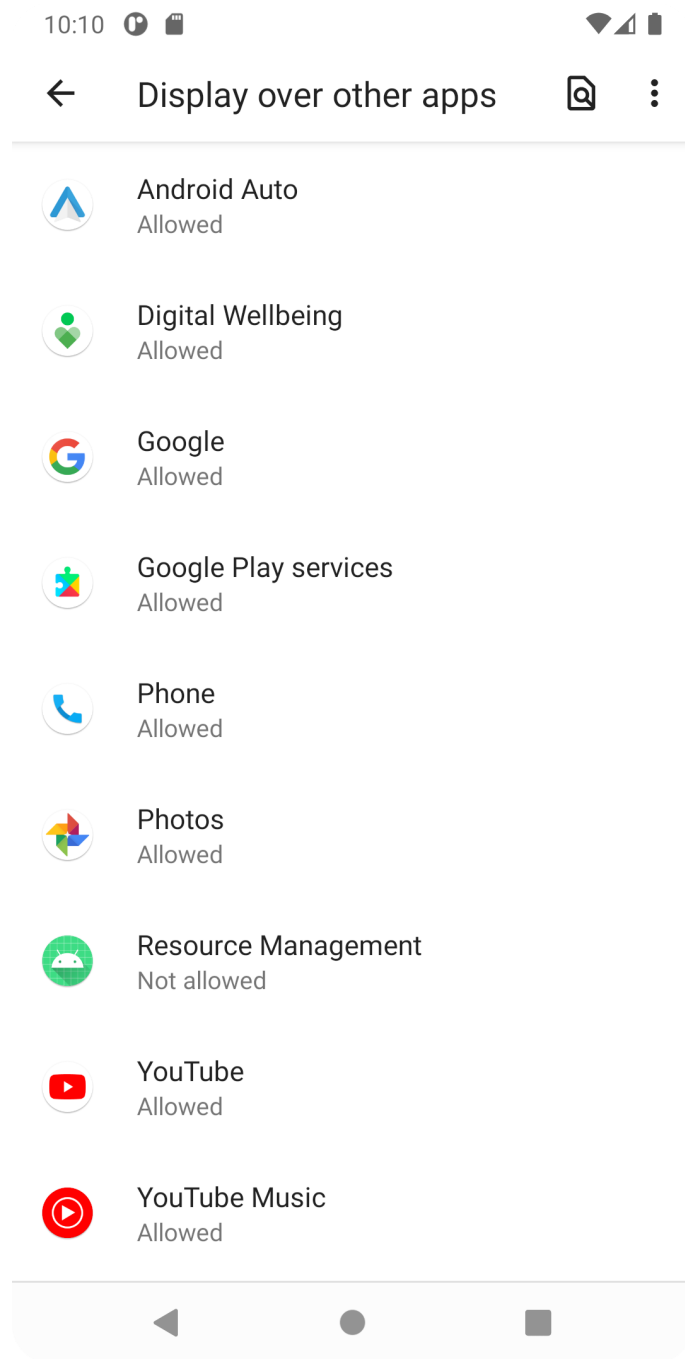


Figura 9: Permissão parte 1

Na tela da figura 10, basta conceder e voltar ao aplicativo.

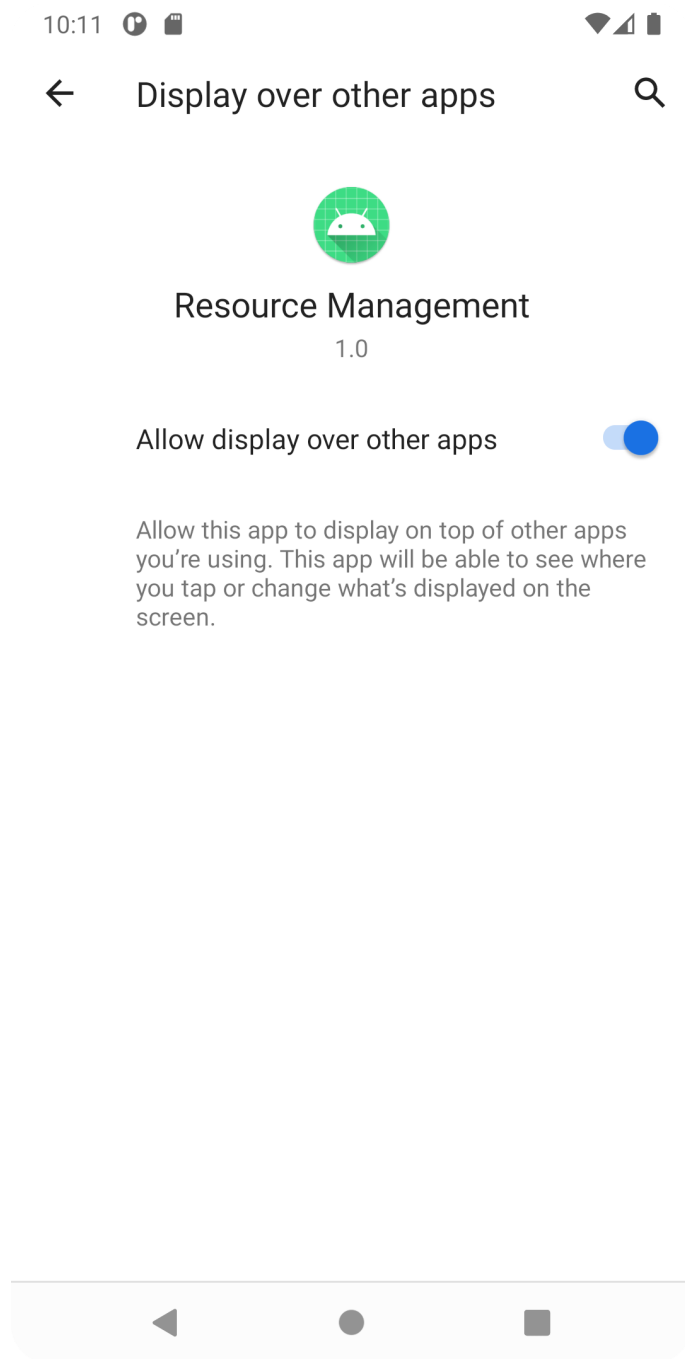


Figura 10: Permissão parte final

2.2.3 Layout

Todos os arquivos .xml de Layout utilizados estão definidos na captura de tela da figura abaixo.

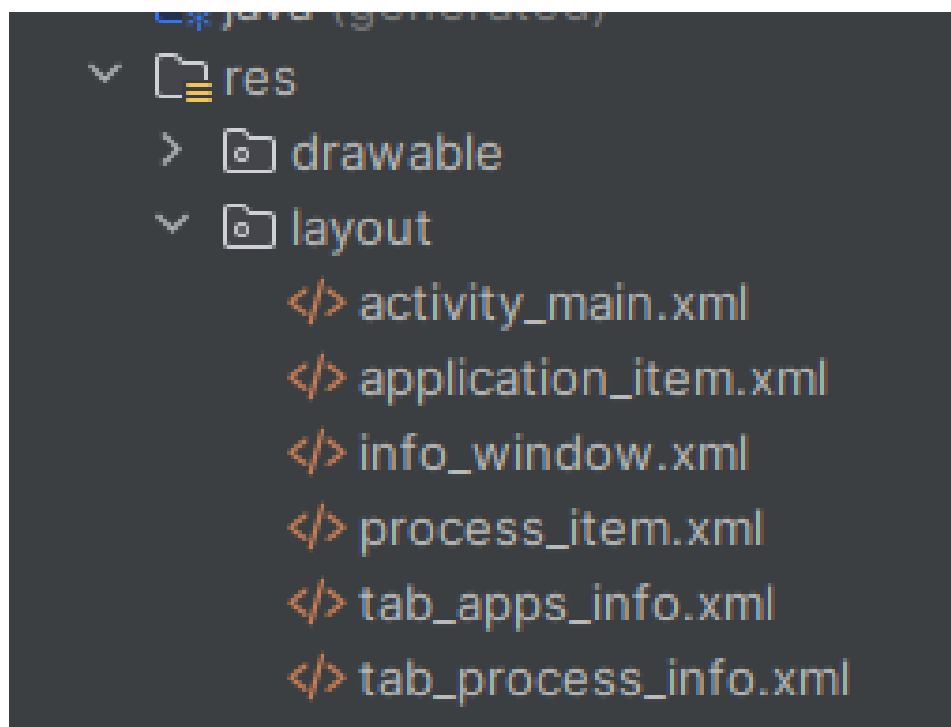


Figura 10: Layout